

Travail

- Créer un répertoire dans lequel vous allez copier le fichier **matériel**.
- Transformer le fichier **matériel** pour que toutes les informations concernant une machine se trouvent sur une seule ligne. Chaque champ d'information étant séparé des autres par le symbole ':'. Le fichier **matériel** transformé sera rangé dans le fichier **mat**.

1. Donner la commande permettant de trier le fichier **mat** par ordre alphabétique sur le nom de la machine.

```
sort -t ':' -k1 mat.txt
```

sort → Trie les lignes du fichier.

-t ':' → Définit : comme séparateur de champ.

-k1,1 → Trie uniquement sur la **première colonne** (nom de la machine).

2. Donner la liste des machines avec leur position respective.

```
awk -F ':' '{print $1, $NF}' mat.txt
```

awk : Outil puissant pour traiter et manipuler du texte.

-F ':' : comme séparateur de champ

\$1 : Correspond au premier champ (le nom de la machine).

\$NF : Correspond au dernier champ de la ligne (NF signifie "Number of Fields", donc \$NF est toujours le dernier champ).

print \$1, \$NF : Affiche le premier champ (nom de la machine) suivi du dernier champ (position de la machine).

3. Donner la liste des noms de machine par modèle.

```
awk -F ':' '{print $5, $1}' mat.txt | sort
```

awk : Outil utilisé pour extraire et manipuler du texte.

-F ':' : Définit : comme séparateur de champs.

\$5 : Sélectionne le 5^e champ (supposé être le modèle de la machine).

\$1 : Sélectionne le 1^{er} champ (nom de la machine).

`print $5, $1` : Affiche d'abord le modèle, puis le nom de la machine.

`| sort` : Trie les résultats par ordre alphabétique selon le modèle.

4. Donner le nombre de machines par position.

```
awk -F ':' '{positions[$NF]++} END {for (position in positions) print position, positions[position]}' mat.txt
```

`awk` : Commande utilisée pour manipuler des fichiers texte.

`-F ':'` : Définit : comme séparateur de champs.

`positions[$NF]++` :

- `$NF` représente le dernier champ de chaque ligne (ici, la position de la machine).
- `positions[$NF]++` incrémente un compteur pour chaque position rencontrée.

`END {for (position in positions) print position, positions[position]}` :

- À la fin du traitement, une boucle `for` parcourt le tableau `positions`.
- Elle affiche chaque position suivie du nombre de machines associées

5. Donner le nombre de machines par modèle.

```
awk -F ':' '{models[$5]++} END {for (model in models) print model, models[model]}' mat.txt
```

`awk` : Utilisé pour traiter et manipuler des fichiers texte.

`-F ':'` : Définit : comme séparateur de champs.

`models[$5]++` :

- `$5` représente le 5^e champ de chaque ligne (supposé être le modèle de la machine).
- `models[$5]++` incrémente un compteur pour chaque modèle rencontré.

`END {for (model in models) print model, models[model]}` :

- Une fois toutes les lignes lues, une boucle `for` parcourt le tableau `models`.
- Elle affiche chaque modèle suivi du nombre de machines associées.

6. Donner la liste des noms de machine et leur numéro IP par ordre croissant du numéro IP. Pour répondre à cette question, une solution consiste à utiliser la commande `sort`. Toutes les machines ont un numéro de classe B dont la partie réseau vaut 172.31, il faudra donc utiliser le troisième et le quatrième octet de l'adresse pour trier la liste des machines.

```
awk -F ':' '{split($2, ip, "."); print ip[3], ip[4], $1, $2}' mat.txt | sort -n -k1,1 -k2,2 | awk '{print $3, $4}'
```

`awk -F ':' '{split($2, ip, "."); print ip[3], ip[4], $1, $2}' mat :`

- `-F ':'` → Définit : comme séparateur de champs.
- `split($2, ip, ".")` → Sépare l'adresse IP (\$2) en un tableau `ip` en utilisant `.` comme délimiteur.
- `print ip[3], ip[4], $1, $2` → Affiche le 3^e et le 4^e octet de l'IP, suivi du nom de la machine et de l'IP complète.

`sort -n -k1,1 -k2,2 :`

- Trie les résultats numériquement (`-n`).
- `-k1,1` → Trie d'abord par le 3^e octet de l'IP.
- `-k2,2` → Trie ensuite par le 4^e octet.

`awk '{print $3, $4}' :`

- Affiche uniquement le nom de la machine et son IP.

7. Donner la liste des machines installées en 1991.

```
awk -F ':' '$8 ~ /91/ {print $1}' mat.txt
```

`awk` : Commande utilisée pour traiter du texte.

`-F ':'` : Définit : comme séparateur de champs.

`$8 ~ /91/` :

→ `$8` correspond au 8^e champ (supposé être l'année d'installation).

→ `/91/` signifie "rechercher les lignes où le 8^e champ contient 91".

`print $1` : Affiche le premier champ (\$1), qui représente le nom de la machine.

8. Donner la liste des machines installées depuis 1992.

```
awk -F ':' '{split($8, a, "/"); if (a[3] >= 92) print $1}' mat.txt
```

awk : Commande permettant de manipuler des fichiers texte.

-F ':' : Utilise : comme séparateur de champs.

split(\$8, a, "/") :

- Sépare le contenu du champ \$8 en plusieurs parties à l'aide du séparateur /.
- Stocke les parties dans le tableau a[].

if (a[3] >= 92) :

- Vérifie si la troisième partie (a[3]) est supérieure ou égale à 92, ce qui signifie une installation en 1992 ou plus tard.

print \$1 : Affiche le nom de la machine (\$1).

9. Donner la liste des machines pour un propriétaire donné en argument (script shell).

```
#!/bin/bash
```

```
awk -F':' -v prop="$1" '$11 == prop {print $1}' mat.txt
```

```
/machinepropro.sh "equipe graphique".
```

```
#!/bin/bash
```

Ceci est le shebang, il indique que le script doit être exécuté avec bash.

```
awk -F':' -v prop="$1" '$11 == prop {print $1}' mat.txt
```

C'est la commande principale qui fait le travail avec awk.

- -F':' → Définit : comme séparateur de champ (les données de mat.txt sont séparées par :).
- -v prop="\$1" → Crée une variable prop qui prend la valeur du premier argument du script (\$1), c'est-à-dire le nom du propriétaire que l'on cherche.
- \$11 == prop → Vérifie si le 11e champ (le propriétaire) est égal à la valeur de prop.
- {print \$1} → Affiche le 1er champ, qui est le nom de la machine.

10. Donner la liste des machines pour un type de machine donné en argument (script shell).

```
#!/bin/bash
```

```
awk -F':' -v type="$1" '$5 == type {print $1}' mat.txt
```

-F':' → Définit : comme séparateur de champ.

-v type="\$1" → Stocke l'argument (\$1) dans une variable type.

\$5 == type → Vérifie si le 5e champ (type de machine) correspond à l'argument.

{print \$1} → Affiche le nom de la machine (champ 1).

./typemachine.sh "station de travail"

11. Donner l'ensemble des informations concernant une machine donnée en argument (script shell).

#!/bin/bash

awk -F':' -v machine="\$1" '\$1 == machine {print}' mat.txt

./info.sh oerbier

- **awk** : Commande pour traiter et filtrer les données.
- **-F':'** : Définit : comme séparateur de champ
- **-v machine="\$1"** :
 - **-v** permet de définir une variable dans awk.
 - Ici, **machine** prend la valeur de \$1 (le premier argument donné au script, c'est-à-dire le nom de la machine).
- **\$1 == machine** :
 - **\$1** représente le premier champ de chaque ligne de mat.txt, qui contient le nom de la machine.
 - On vérifie si ce champ est égal à la valeur donnée en argument (machine).
- **{print}** :
 - Si la condition **\$1 == machine** est vraie, alors awk affiche (print) toute la ligne correspondant à la machine.